

Arduino Cookbook : De la Théorie à la Pratique

Chapitre 6 : Obtenir des données des Capteurs (Sensors)

Présenté par : Dr B. DIOP

29 mars 2026

Référence : Michael Margolis, *Arduino Cookbook*, 2nd Edition, O'Reilly

6.0 Introduction : Donner des sens à l'Arduino

Qu'est-ce qu'un capteur ?

Un capteur (sensor) est un composant électronique qui convertit un phénomène physique (lumière, chaleur, pression, ondes sonores) en un signal électrique mesurable par le microcontrôleur.

Dans ce chapitre, nous allons apprendre à mesurer et exploiter :

- **La lumière** : Photorésistances (LDR).
- **Le mouvement** : Capteurs Infrarouges Passifs (PIR).
- **La distance** : Sonars à ultrasons et télémètres infrarouges.
- **Les vibrations et le son** : Éléments piézoélectriques et microphones.
- **La température** : Capteurs analogiques linéaires (TMP36).
- **L'identification** : Puces RFID.

6.1 Détecter la lumière (Photorésistance / LDR)

Principe physique : Une résistance dépendante de la lumière (LDR) voit sa résistance chuter quand elle est éclairée (de $1\text{ M}\Omega$ dans l'obscurité à $\sim 10\text{ k}\Omega$ en pleine lumière).

Le Pont Diviseur de Tension : L'Arduino ne mesure pas la résistance, il mesure la tension. Il faut donc associer la LDR à une résistance fixe (souvent $10\text{ k}\Omega$) pour créer une tension variable lisible par la broche analogique.

[Insérer ici le schéma du circuit LDR - Pont diviseur]

Code : Lecture de la luminosité ambiante

```
const int capteurLumierePin = A0;
const int ledPin = 9; // LED pour reagir a la lumiere

void setup() {
  Serial.begin(9600);
  pinMode(ledPin, OUTPUT);
}

void loop() {
  int luminosite = analogRead(capteurLumierePin);
  Serial.println(luminosite);

  // Si la piece est sombre (valeur faible), on allume la LED
  if (luminosite < 300) {
    digitalWrite(ledPin, HIGH);
  } else {
    digitalWrite(ledPin, LOW);
  }
  delay(100);
}
```

6.2 Détecter le mouvement (Capteur PIR)

Capteur Infrarouge Passif (PIR) :

- **Fonctionnement** : Détecte les variations de rayonnement infrarouge (la chaleur corporelle) en mouvement.
- **Avantage** : Très facile à utiliser. Il renvoie simplement un signal numérique : HIGH (mouvement détecté) ou LOW (aucun mouvement).
- **Réglages** : Souvent équipés de potentiomètres matériels pour régler la sensibilité et le délai de maintien (Time Delay).

[Insérer ici l'image d'un module PIR standard]

Code : Alarme de mouvement avec PIR

```
const int pirPin = 2;    // Broche connectee a la sortie du PIR
const int alarmePin = 13;

void setup() {
    pinMode(pirPin, INPUT);
    pinMode(alarmePin, OUTPUT);
    Serial.begin(9600);

    Serial.println("Calibration du capteur PIR (30s)...");
    delay(30000); // Le capteur a besoin de temps pour analyser l'infrarouge ambiant
}

void loop() {
    int mouvement = digitalRead(pirPin);

    if (mouvement == HIGH) {
        digitalWrite(alarmePin, HIGH);
        Serial.println("Intrusion detectee !");
    } else {
        digitalWrite(alarmePin, LOW);
    }
}
```

6.3 Mesurer la distance (Capteur Ultrason - HC-SR04)

Principe du Sonar : Le capteur envoie une impulsion sonore inaudible pour l'homme (Ultrason) via l'émetteur (Trig). Il chronomètre le temps que met l'écho pour rebondir sur un obstacle et revenir au récepteur (Echo).

La formule mathématique :

$$Distance = \frac{Temps \times VitesseDuSon}{2}$$

(On divise par 2 car le son fait un aller-retour).

[Insérer ici le schéma de rebond ultrasonique]

Code : Utilisation native du HC-SR04

```
const int trigPin = 9;
const int echoPin = 10;

void setup() {
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  Serial.begin(9600);
}

void loop() {
  // 1. Generation de l'impulsion (10 microsecondes)
  digitalWrite(trigPin, LOW); delayMicroseconds(2);
  digitalWrite(trigPin, HIGH); delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  // 2. Mesure de la duree de l'echo
  long duree = pulseIn(echoPin, HIGH);

  // 3. Conversion en centimetres (Vitesse du son ~ 340 m/s)
  long distance = duree * 0.034 / 2;

  Serial.print("Distance : "); Serial.print(distance); Serial.println(" cm");
  delay(100);
}
```


6.4 Mesurer la distance (Capteur Infrarouge Sharp)

Télémètre Infrarouge (Ex : Sharp GP2Y0A21YK) :

- Contrairement aux ultrasons, il utilise la réflexion de la lumière infrarouge pour déterminer la distance (Triangulation optique).
- **Avantage** : Pas perturbé par les échos acoustiques ou le vent.
- **Inconvénient** : La sortie est analogique et ****non-linéaire****. La tension chute de façon logarithmique à mesure que la cible s'éloigne. Il faut une formule mathématique complexe ou une table de correspondance pour convertir la tension en centimètres.

6.5 Détecter des vibrations (Capteur Piézoélectrique)

Le composant Piézo : Un disque de céramique qui génère une petite tension électrique lorsqu'il est déformé ou frappé ("Knock sensor").

Problème électrique : Les pics de tension peuvent dépasser 5V et endommager l'Arduino. Il faut le monter en parallèle avec une résistance de forte valeur (1 M Ω) et idéalement une diode Zener de protection.

```
const int piezoPin = A0;
const int seuil = 100; // Sensibilite a ajuster

void loop() {
    int lecture = analogRead(piezoPin);

    if (lecture > seuil) {
        Serial.println("Choc ou vibration detectee !");
        delay(50); // Anti-rebond mecanique
    }
}
```

6.6 Détecter le son (Microphone analogique)

L'acquisition sonore sur Arduino : L'Arduino n'est pas assez puissant pour traiter des fichiers audio complexes, mais il excelle pour réagir au ****volume**** sonore ambiant (ex : pour faire un VU-mètre ou réagir à un claquement de mains).

- Le signal audio est une onde alternative (AC). Il oscille autour de 0V (ex : -1V à +1V).
- L'Arduino ne peut lire que des tensions positives (0 à 5V).
- Les modules microphones pour Arduino intègrent un amplificateur opérationnel qui décale le signal à 2.5V pour éviter les valeurs négatives.

6.7 Mesurer la température avec précision

Le capteur analogique TMP36 / LM35 :

- Très différent d'une thermistance classique (qui nécessite un étalonnage complexe).
- Ces capteurs génèrent une tension proportionnelle et linéaire à la température.
- ****Formule du TMP36 :*** Il délivre 750mV à 25°C, et la tension varie de 10mV par degré Celsius. Il commence à 500mV pour 0°C (pour pouvoir lire les températures négatives).

[Insérer l'image du TMP36 et ses 3 broches : VCC, Vout, GND]

Code : Convertir la tension du TMP36 en Degrés

```
const int tempPin = A0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  int lectureAnalogique = analogRead(tempPin);

  // Conversion en tension (Volts)
  float tension = lectureAnalogique * 5.0 / 1024.0;

  // Formule specifique au TMP36 : (Tension en mV - 500) / 10
  float temperatureC = (tension - 0.5) * 100.0;

  Serial.print("Temperature : ");
  Serial.print(temperatureC);
  Serial.println(" *C");

  delay(1000);
}
```

6.8 Identification et Accès : Lire les badges RFID

RFID (Radio Frequency Identification) :

- Permet à l'Arduino d'identifier des objets ou des personnes via des badges/cartes sans contact.
- Le module RFID (Lecteur) génère un champ électromagnétique qui alimente la puce passive contenue dans la carte.
- La carte renvoie alors son code unique (UID).
- La communication avec l'Arduino se fait généralement via le protocole série (UART) ou le bus SPI (ex : module RC522 très populaire).

Logique logicielle d'un contrôle d'accès RFID

(L'implémentation exacte dépend du modèle de lecteur RFID)

```
// Logique simplifiée d'identification
String badgeAutorise = "4A FB 2C 80";

void loop() {
  if (nouveauBadgeDetecte()) {
    String uidLu = lireUID(); // Recupere le code de la carte

    if (uidLu == badgeAutorise) {
      Serial.println("Acces Autorise !");
      ouvrirPorte(); // Active un relais ou un servomoteur
    } else {
      Serial.println("Acces Refuse.");
      declencherAlarme();
    }
  }
}
```

Sécurité

Sur les systèmes basiques, l'UID peut être cloné. Pour une vraie sécurité, il faut utiliser des blocs de mémoire cryptés dans la carte MIFARE.

Résumé du Chapitre 6

- **Calibration** : La plupart des capteurs analogiques nécessitent une phase de calibration au démarrage (pour établir le minimum et le maximum ambiants).
- **Filtrage du bruit** : Un capteur brut est souvent instable. Utilisez des moyennes (prendre 10 mesures et les diviser par 10) pour stabiliser vos affichages de température ou de distance.
- **Linéaire vs Logarithmique** : Faites attention à la courbe de réponse de vos capteurs. Un LM35 est linéaire (facile), une thermistance classique ne l'est pas.
- **Bibliothèques** : Pour les capteurs complexes (RFID, Ultrasons avancés), utilisez toujours les librairies existantes (ex : NewPing pour l'HC-SR04) pour optimiser les performances de votre code.

Fin du Chapitre 6. Dans le prochain chapitre, nous utiliserons ces données pour interagir avec l'utilisateur via des affichages visuels (LEDs, LCD, Matrices).